

Build your Python foundation: *environment*, thinking, and your first data script

Work through every section before moving on — environment isolation, problem decomposition, and real data output are prerequisites, not optional steps.

NAME

DATE

SESSION

1 Audit your current environment

- Write down your Python version, package manager, and where packages live. Run: `python --version · pip list · which python`
- Do you have isolation? (`venv` / `conda` / `pyenv`) If not — set it up now. This is your first step, not optional.
- How would you revert your environment if something breaks? Write the exact steps — before disaster, not after.
- **Why it matters:** isolation prevents "code works today, breaks next month" — the most common beginner failure mode in production ML.

NOTES & ANSWERS

2 Decompose a real problem you face

- Pick *one* real data problem: analyzing a spreadsheet, understanding a dataset, processing logs, or calculating metrics from records.
- **State the big problem** in one sentence → break it into 3–4 smaller pieces → decide which piece to solve first.
- Write this decomposition *before* opening Python. Computational thinking starts on paper — not in a REPL.

PROBLEM DECOMPOSITION

3 Write your "hello data" script

- Create a script that does all three: (1) loads real data (CSV / JSON / dict / API) · (2) calculates one meaningful result · (3) saves output. Run it.

```
# 1. Load → import csv / json / requests; open your real file
# 2. Result → result = sum(row['value'] for row in data)
# 3. Save → open('output.txt', 'w').write(str(result))
```

OUTPUT & OBSERVATIONS

4 Break your code & read the error

- Introduce one deliberate bug. Run the script. Copy the *exact* error below — line number, error type, message. Repeat three times.

```
# Bug 1 (wrong key) → data['nonexistent_key']
# Bug 2 (type error) → total = count + "records"
# Bug 3 (your choice) →
```

ERROR MESSAGES (COPY EXACTLY)

5 Verify your foundation

- Before moving forward, confirm each: ☐ I can explain why environment isolation prevents future pain · ☐ I can explain decomposition in my own words · ☐ My script produced real output from real data.
- If yes to all three — you've built the foundation. If not: **name the piece that needs another cycle and go back.** Skipping makes the next module harder, not easier.

SELF-ASSESSMENT